

Grater

- will my change break anyone who depends on me?

Mentored by: [Pablo Baeyens](#) [Damien Mathieu](#)

Author: Amisha Chhajer

Date: 04/03/2026 - 30/05/2026

Abstract

Project Idea/Tracking Issue: [Grater](#)

The objective of this project is to develop a tool in GO (Grater) inspired by [Rust's Crater](#) to automate the detection of regressions introduced by a change by running the tests of its downstream dependents and inspecting it for failures over the new code. This project would be a part of [OpenTelemetry Go Build tools](#).

Objectives/Deliverables

Background

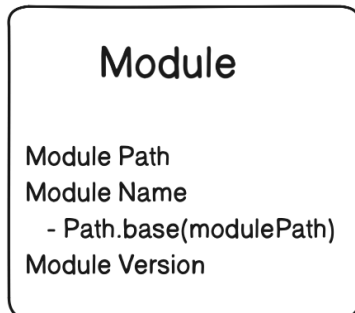
We intend to create a tool which would be used as a part of our deployment process, running tests of our dependents to catch regressions caused before the release of a new version. The tool should also be able to handle the regression testing for release of two module's new versions, for example, running tests for module A with unreleased module B v2 injected in the dependents instead of module B v1, when we release new versions of two new modules simultaneously.

Core Objectives

- Add dependents manually in case we don't intend to use automated find.
- Replace dependencies of any module.
- Run tests of all the dependents over the base and head version of a module.
- Generate a report of the results and regressions(if any) with the module name, version and Output log.
- Make the tool usable from the command line.

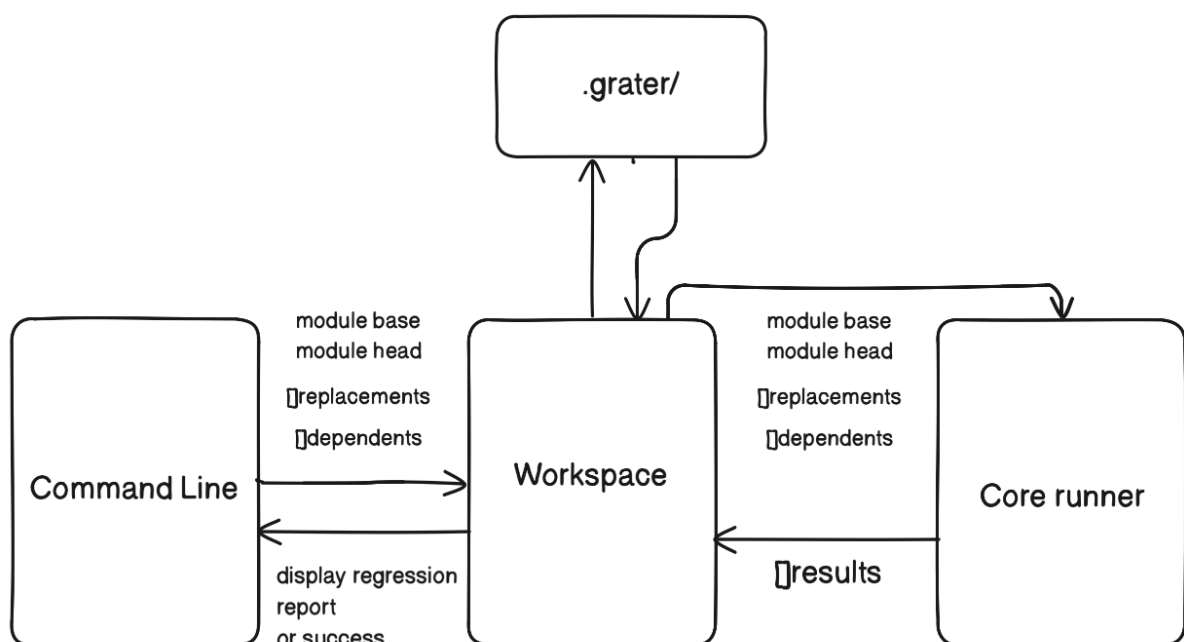
Architecture And Implementation

Module



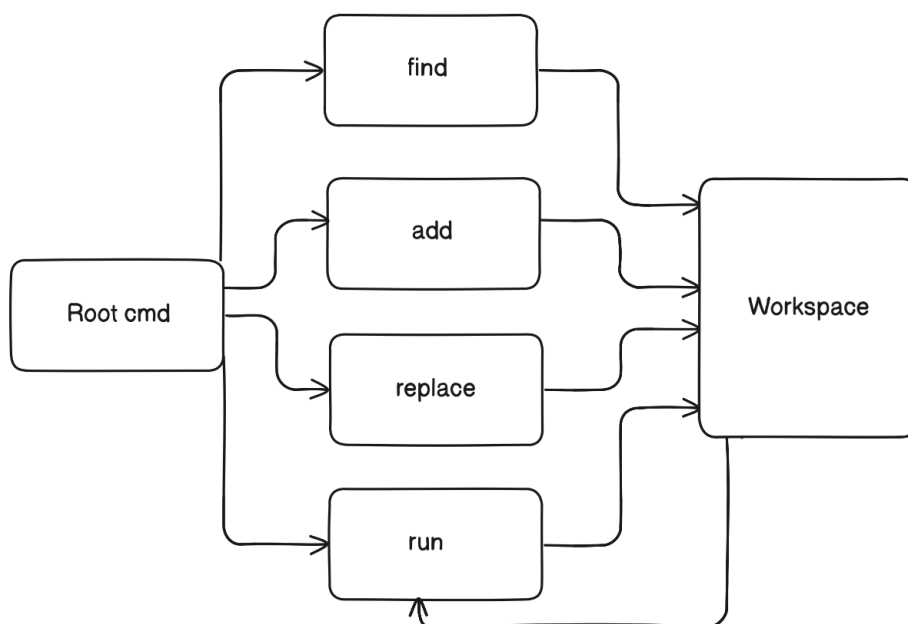
- Module is the most basic unit of our Architecture.
- A module can represent a main module, dependent or a replacement.
- Module path can be local or remote.
- Currently we only support modules paths that are either local or remote paths similar to remote paths in go.mod, for example which can be fetched from the proxy.

Workspace

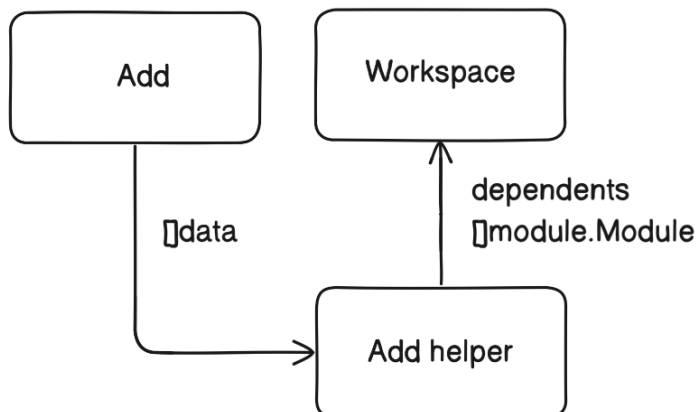


- Workspace has two core responsibilities
 - Receive information (module arrays and modules) from command line and results of tests from Core runner and persist them in disk.
 - Retrieve the persisted information from the `.grater/` directory and pass it to the core backend to execute the testing process, or to the command line to display them.
- Workspace acts as our main communication line between command line and runner backend which facilitates exchange of all information between the two, and saves the data for persistent in desired format, writing data to the disk.

Command Line (cmd)

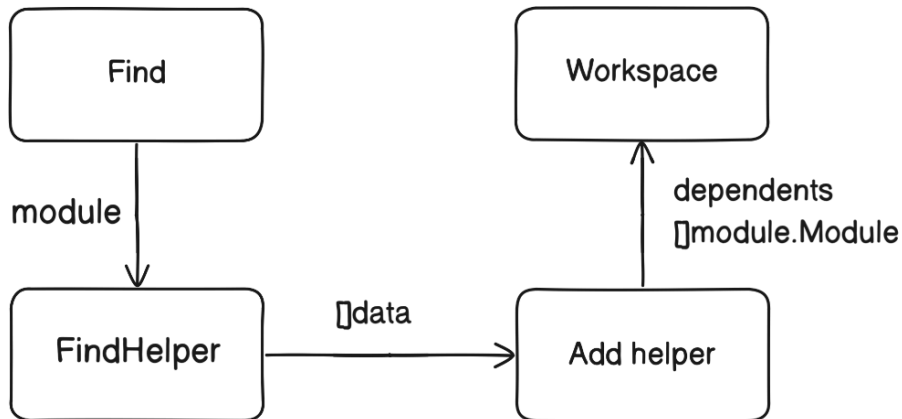


Add



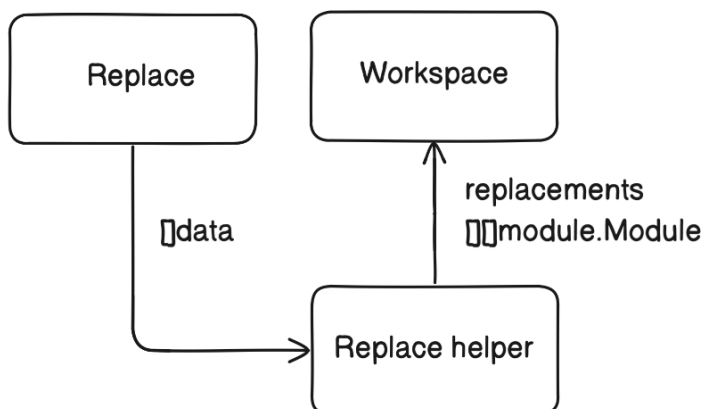
- Add command can add dependents either from local path or by specifying a remote path.
- It internally calls addhelper which formats the data into modules and gives it to workspace to persist in a dependents.json file.

Find (Not Implemented/Ideation)



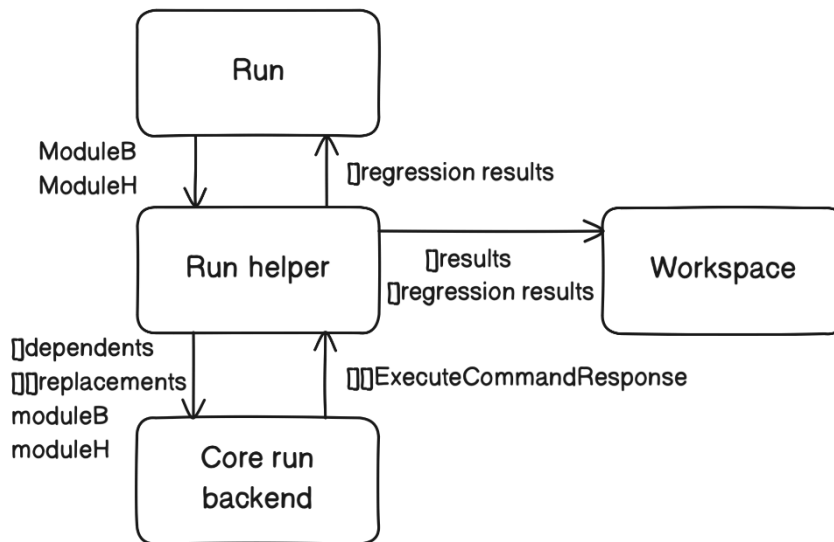
- Find internally calls findhelper that queries pkg.go.dev to fetch dependents of a module.
- findhelper formats the queried result into an array of data that addhelper expects.

Replace



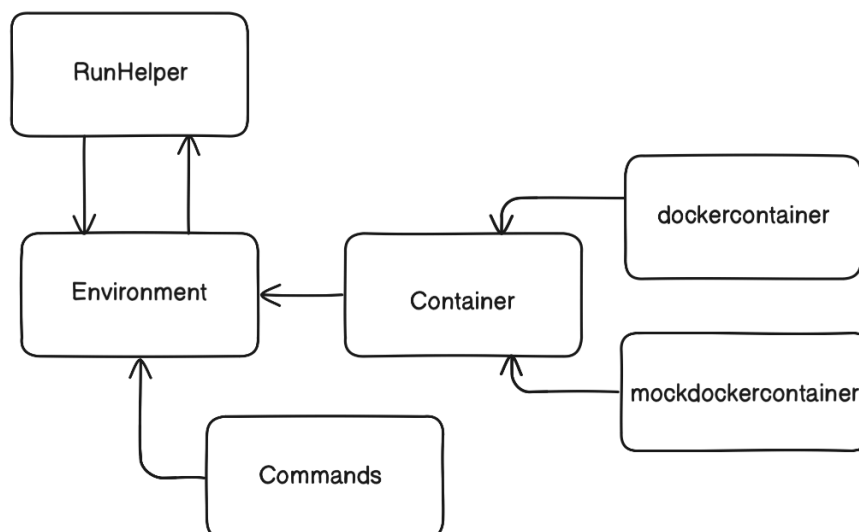
- Replace commands adds replacements in the form [moduleOld, moduleNew].
- Calls replace helper internally to pass it to workspace to persist.

Run



- Run command line calls runHelper which calls Core Run Backend to run tests for module base and module head.
- runHelper passes all the required data like dependents and replacements from the workspace to the core run backend.
- runHelper receives the results for the executed tests and returns them to the workspace to persist.
- runHelper returns and displays a regression result report on the command line.

Core Run Backend



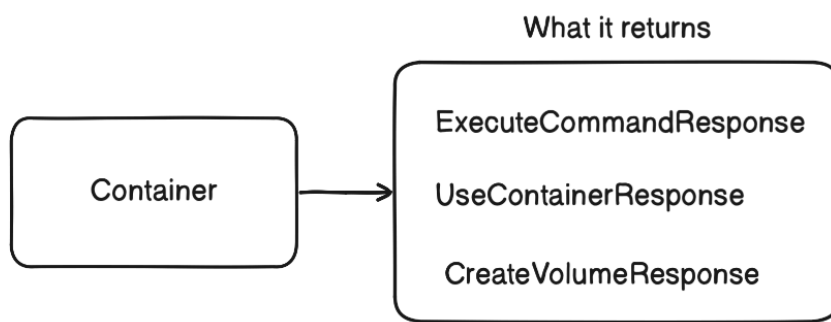
DockerContainer

- Generates an instance of a docker container that interacts with docker daemon using go sdk for docker.
- Implements container Interface.

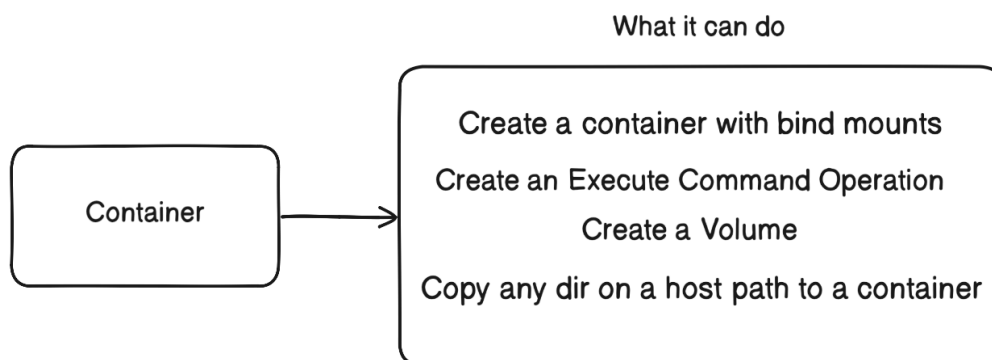
MockDockerContainer

- Mock implementation of container interface that does not call docker daemon, makes no real operation.
- Useful for slow testing when we want to assert methods are called but cannot run actual integration tests.

Container



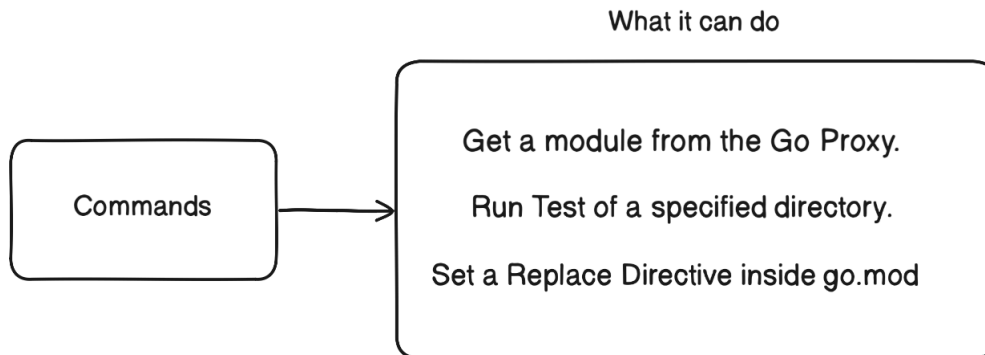
- **ExecuteCommandResponse:** returns Output and ExitCode of the command we executed.
- **UseContainerResponse:** returns Cleanup() that destroys the created container, ContainerID.
- **CreateVolumeResponse:** returns Cleanup(), that destroys the created volume.



- **CreateContainer:** creates a container, with the option to bind mounts.
- **CreateVolume:** Creates a Volume.

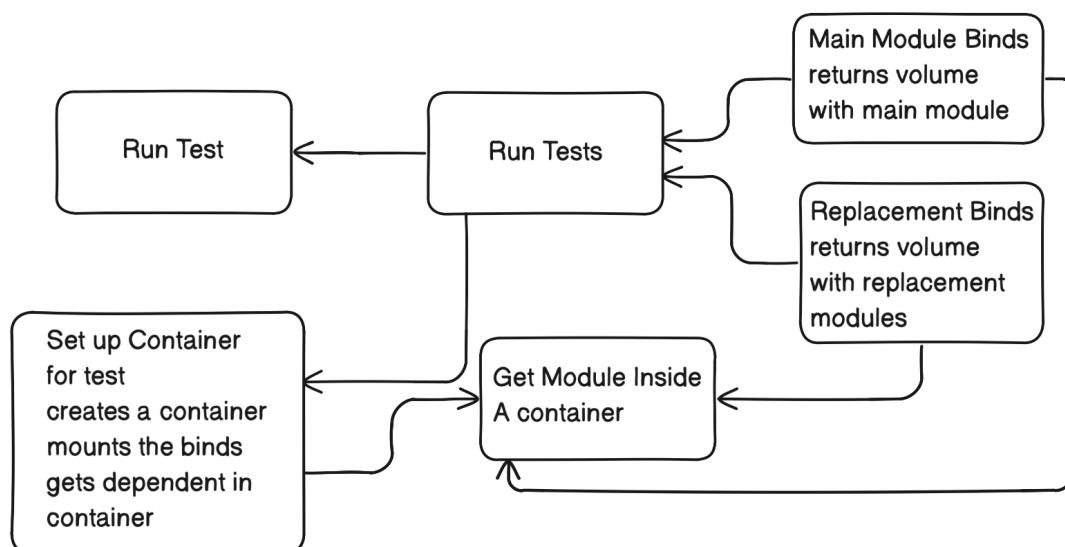
- **ExecuteCommand:** Executes a command inside a specified container with the option of specifying a working directory.
- **CopyToContainer:** Copies a Dir with a local path inside a container.

Commands



- commands is an abstraction for frequently used commands inside environment.
- **GetModuleFromProxy:** Gets a module from the Go Proxy and copies it to the given module path.
- **RunTest:** executes the command go test ./... inside the specified modulePath and returns an ExecuteCommandResponse.
- **SetReplaceDirective:** adds a replace directive inside the go.mod file at the given path with specified oldRef and newRef.

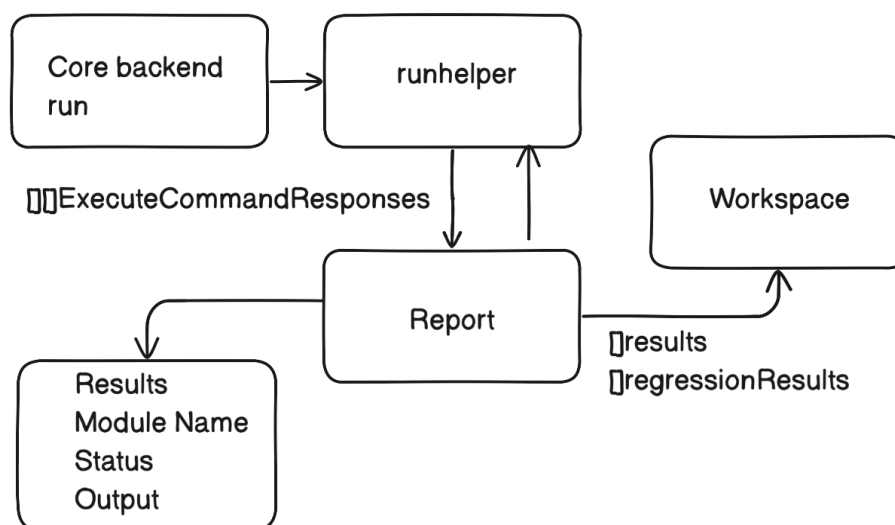
Environment



- **getModuleInsideContainer:** Checks if a module is remote or not, gets it from proxy if its a remote otherwise makes a copy inside the container. Uses path /mainModule/ to store all main modules.

- **getMainModuleBinds:** Runs `getModuleInsideContainer` for `mainModuleHead` for a container that has a volume 'main_module_volume' binded inside it, it is assumed that module base won't be required as dependents already depend on it and would download it the standard way while running tests, and we can use `replace` directive to switch module base to module head later, so we only return `bind` for module head. Uses path `/mainModule/` for storing `mainModule`.
- **getReplacementModuleBinds:** Runs `getModuleInsideContainer` for module `replacementPair[1]` for container that has a volume 'replacements_volume' binded inside it, it is assumed that `replacementPair[0]` won't be required as dependents already depend on it and would download it the standard way while running tests, and we can use `replace` directive to switch `replacementPair[0]` to `replacementPair[1]` later, so we only return `bind` for replacements. Uses path `/replacements/` for storing them.
- **setUpContainerForTest:** This creates a container and mounts the binds provided by `RunTests`, runs `getModuleInsideContainer` for a dependent and stores it in `/dependent/`.
- **RunTest:** Runs test of the dependent using commands, sets a `replace` directive to switch to head, runs test again.
- **RunTests:** Calls `getMainModuleBinds` and `getReplacementModuleBinds` and merges the map into a single map called `binds`, `mainModule` and `replacements` are common for each container as we use them to only inject and not make any operations on them so they are initialised only once at the start. `RunTests` calls `setUpContainerForTest` for each individual dependent then calls `RunTest` with the returned container. Returns an array of `ExecuteCommandResponse` for each base and head run of format `[][]ExecuteCommandResponse`.

Report



- Report takes `[]ExecuteCommandResponses` from runhelper and classifies them into following targets,

BaseResult	HeadResult	Status
1	0 or 1	Broken
0	0	Pass
0	1	Regression

- Creates a `[]Results` array and passes it to workspace for persistence and runhelper for displaying it to the command line.

CI

Tests involving docker are flaky due to network calls, and slow due to module fetching, test running and docker operations, to ensure the other modules remain un-affected by our changes we have made the following enhancements.

- We have updated the CI to run tests only for the modules whose files have changes, we climb up the dirs of files with changes and run tests for each of them.
- We made a separate integration tests build tag that runs docker tests only when integration build tag is used.
- Fixed Codecov to display coverage for individual tools.

Objectives Achieved And Statistics

Core Objectives

- Add dependents manually in case we don't intend to use automated find.
- Replace dependencies of any module.
- Run tests of all the dependents over the base and head version of a module.
- Generate a report of the results and regressions(if any) with the module name, version and Output log.
- Make the tool usable by the command line.

Statistics

- Lines of code: 4000+
- Reduction in CI run time by $\approx 35.79\%$ since the enhancements we made for the CI even after the introduction of time-consuming integration tests.

Before

Actions Performance Metrics

1/1/2026 - 3/31/2026 ▾

Showing data from 1/1/2026 to 3/31/2026

Avg job run time 1m 35s Average run time of jobs in this repository for custom	Avg job queue time 23s Average queue time of jobs in this repository for custom	Job failure rate 9% Failure rate across jobs in this repository for custom	Failed job usage 979 Total minutes used across failed jobs in this repository for custom
---	--	---	---

After

Actions Performance Metrics

4/1/2026 - 5/25/2026 ▾

Showing data from 4/1/2026 to 26 minutes ago

Avg job run time 1m 1s Average run time of jobs in this repository for custom	Avg job queue time 34s Average queue time of jobs in this repository for custom	Job failure rate 9% Failure rate across jobs in this repository for custom	Failed job usage 814 Total minutes used across failed jobs in this repository for custom
--	--	---	---

- Number of PRs: 21
[PRs merged and in review to be merged.](#)

Future Work/Currently Not Supported

- If a dependent depends on a local go module on a host computer currently there is no way to add its support via the command line, this had to be hardcoded for a test using testdata/ which is present locally, however support for adding something like this might be a consideration in the future.
- GetModuleFromProxy is the only supported module fetching method present currently which can be extended to GetModuleFromUrl and many different methods to get a module.
- We can only AddFromFile via a .txt file, having the support of .json and various other file types in addition to a .txt would be a good to have.
- Test coverage can be improved and all failure paths can be tested.
- Support for mainModule being a singular host module with branches that need to be checked out in order to access head is not currently supported but can be easily extended.
- Find dependents of a module by querying [pkg.go.dev](#).
- Adding a new command 'grater clean' that re-initialises the workspace(dir .grater/) to clean the contents of the previous run as currently we need to manually delete the files of the previous run.

- There are multiple ways tests could be flaky, one of them is flakiness via. network calls, like our tests for integration, we have to explicitly extend the API for UseContainer to turn off network calls or handle flaky tests by re-running the test job 3 times if it fails.
- Currently not all dependents that we add have a valid go.mod inside of them at the path that they specify as the dependent, which causes errors like the ones mentioned below when we try to get the module from proxy, hence we cannot run tests for them, this is not a direct risk to the accuracy as this does not give any false positives but does give a lot of false negatives and broken modules which we can essentially utilise by finding a workaround to build and run tests for them.

```
# ./...  
pattern ./...: directory prefix . does not contain main module or its selected d  
ependencies  
FAIL    ./... [setup failed]  
FAIL
```

My Takeaways And Learnings

This was my first time building an architecture and owning the development E2E, with great responsibility also came great learning. I was incredibly surprised by how much I was able to build and achieve in the span of a few months with a new language, new ecosystem and new concepts. I had never worked with Go, docker, Go patterns, and this project challenged all those very deeply, including how Go handles dependencies, and several edge cases that I had to figure out, how docker stores data, how containers and volumes are managed.

My mentors were incredibly patient and supportive throughout, they were always my greatest enablers and the credit of successful completion of this project goes entirely to them, beyond the immediate knowledge of this project I have also learned great software development skills, contributing to OpenTelemetry had always been a dream which I was able to accomplish with their help.

It was always a dream to design and build a product which would eventually be used by people. I was able to accomplish this personal goal. Taking design decisions for a project, weighing all the pros and cons of several approaches and building something was an incredible experience.